Computational costs of smoothness-indexed penalized least squares: spectral reuse, cached grids and solver trade-offs

Heriberto Espino-Montelongo[a], Daniela Cortés-Toto[a]

[a] *Universidad de las Américas Puebla, San Andrés Cholula, Puebla, Mexico*

ARTICLE INFO

Keywords:
Smoothness index
penalized least squares
algorithmic complexity
caching
spectral decomposition
computational benchmarking
cross-validation

ABSTRACT

Finite-difference penalized least squares is most often tuned through a penalty λ whose numerical scale varies with series length and difference order. We study the computational cost of using instead normalized trace smoothness $s \in [0,1]$ for any $1 \le d < n$ and a fixed, possibly nonzero, mean of the penalized differences. A singular-value representation gives an invertible $s \leftrightarrow \lambda$ map and permits caching of spectra, data projections and grids across criteria and independent series. We distinguish single fitted curves, single criterion scores, repeated full-grid searches and multi-series reuse, and derive arithmetic and memory complexity for dense solves, dense Cholesky, general banded solves, banded Cholesky, dense eigendecomposition and SVD. We analyze an archived 80,640-row paired Windows benchmark with seven numerical backends, five criteria, eight search policies, two fixed means, two sample sizes and two seeds. In a representative $n = 48, d = 2$ five-criterion experiment, repeated native-smoothness mapping took 11.234 ms, against 3.980 ms when a mapped grid was shared, a descriptive 2.82-fold reduction. Native-s and pretranslated searches select identically when they evaluate identical penalties; an independent logarithmic penalty grid is not the same statistical candidate set. Separate repeated-fit and 20-seed solver benchmarks show that the fastest method depends on workload and numerical-validity requirements. Timings measure the archived machine and are not universal complexity proofs. This computational companion will ultimately be combined with a separate high-precision study to characterize accuracy–runtime trade-offs.

1. Introduction and smoothness coordinate

Quadratic finite-difference penalized least squares (PLS) has a long history in Whittaker–Henderson smoothing (Whittaker, 1923; Guerrero, 2007). The penalty λ is mathematically convenient, but the same numerical value need not express the same smoothness for different difference orders or sample lengths. Guerrero's trace-based smoothing measure (Guerrero, 2008) and its use with CV, GCV, AICc and BIC (Cortés-Toto, Guerrero and Reyes, 2017) are established precedents. We do not claim to invent the smoother or its tuning criteria; instead, we investigate the *computational cost of an interpretable bounded coordinate and the reuse of its mapping across many evaluations.*

For $y \in \mathbb{R}^n$, difference order $1 \le d < n$, and fixed mean of the penalized differences μ, let $D_d \in \mathbb{R}^{(n-d)\times n}$ denote forward differences and define

$$\widehat{\tau}_{\lambda,\mu} = \arg\min_{\tau} \{\|y-\tau\|_2^2 + \lambda\|D_d\tau - \mu\mathbf{1}\|_2^2\}. \quad (1)$$

Put $r = n-d$, $Q_d = D_d^\top D_d$, and $H_\lambda = (I+\lambda Q_d)^{-1}$. A polynomial particular solution is $p_{\mu,t} = \mu\binom{t-1}{d}$, and $\widehat{\tau} = p_\mu + H_\lambda(y-p_\mu)$. The penalty has a d-dimensional polynomial nullspace, so $\mathrm{tr}(H_\lambda) \to d$ as $\lambda \to \infty$.

ORCID(S):

Write $\delta_1, \dots, \delta_r > 0$ for the positive eigenvalues of Q_d. We use the normalized smoothness index

$$S_d(\lambda) = \frac{n - \mathrm{tr}(H_\lambda)}{n-d} = \frac{1}{r}\sum_{j=1}^{r} \frac{\lambda\delta_j}{1+\lambda\delta_j}. \quad (2)$$

Each $\delta_j > 0$ implies $S_d'(\lambda) = r^{-1}\sum_j \delta_j/(1+\lambda\delta_j)^2 > 0$, hence S_d is continuous, strictly increasing, and maps $[0,\infty]$ onto $[0,1]$. Every interior s has a unique penalty $\Lambda_{n,d}(s)$. The map depends only on (n,d), *not* on observations or the fixed reference mean μ. Therefore it can be cached across different series, criteria and μ values. The denominator $n-d$ normalizes an established trace-based index; it is not a new statistical model.

1.1. Shift-free full spectral reconstruction

Let $D_d = U\,\mathrm{diag}(\sigma)V^\top$ be a reduced SVD over the r positive singular modes and write $w_j = \lambda\sigma_j^2/(1+\lambda\sigma_j^2)$. A numerically useful identity is

$$\widehat{\tau}_{\lambda,\mu} = y - V\big[w \odot (V^\top y) - \mu(w \oslash \sigma) \odot (U^\top \mathbf{1})\big], \quad (3)$$

where multiplication and division are componentwise. It avoids forming the potentially enormous polynomial p_μ at high d and costs $O(nr)$ for a complete curve once the SVD is available. A separate high-precision companion manuscript evaluates its numerical accuracy. Here we study *runtime costs* without treating a numerically invalid fast method as a winner.

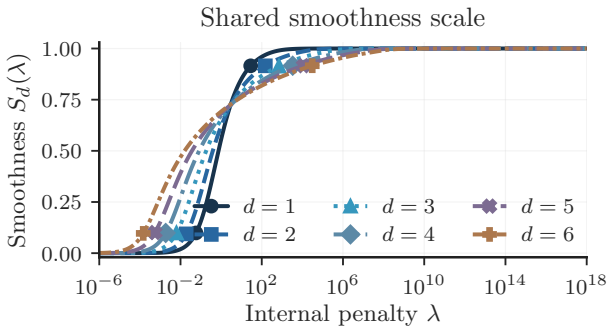


Figure 1: Smoothness versus penalty for $n = 48$, orders $d = 1, \ldots, 6$. The sixth-order line illustrates the mathematical mapping, not a completed Windows timing benchmark for $d = 6$.

The exact endpoints are interpolation at $s = 0$ and the polynomial-constrained limit at $s = 1$, which is evaluated analytically rather than factorizing a matrix at $\lambda = \infty$. Figure 1 illustrates why a user benefits from a common unit-interval smoothness coordinate even though the corresponding penalty scales vary with d.

1.2. Research questions

A single fitted curve, one criterion score, a complete grid search, and R independent searches are different workloads. We distinguish: setup costs; full fitted-trend reconstruction; trace and leverage evaluation for CV/GCV/AICc/BIC/MLE; one-point evaluation; matched 17-point grids; locally optimized continuous searches; and repeated-series reuse. Spectral preprocessing may be expensive for one query yet efficient when amortized. The question is not whether s magically lowers the cost of optimizing λ, but whether the *same exact set* $\Lambda_{n,d}(s_j)$ can be calculated once, reused, and exposed to users as interpretable $s_j \in [0, 1]$.

2. Algorithmic time and storage complexity

Let n be series length, d difference order, $r = n - d$, K penalty candidates, R observed series, C scoring criteria and I root-solver evaluations per requested s. All big-O statements are arithmetic models, not measured wall-clock rates. The penalty matrix $Q_d = D_d^\top D_d$ has half-bandwidth d.

2.1. Factorization and one complete fitted curve

Full SVD of the $r \times n$ difference matrix requires $O(nr^2)$ time and $O(nr)$ storage, or $O(n^3)$ and $O(n^2)$ respectively when d is small relative to n. Dense symmetric eigendecomposition of Q_d also has cubic time and quadratic storage; forming Q_d can exacerbate spectral conditioning. After spectral setup a full fitted curve uses $O(nr)$ products. Dense general solves and dense Cholesky require $O(n^3)$ per new penalty. General banded solving and banded Cholesky can factor in $O(nd^2)$ and use $O(nd)$ storage, followed by $O(nd)$ triangular or back substitution. Thus structured methods can be substantially faster for a *single fit*; Cholesky and general banded LU must not be conflated.

2.2. Criterion evaluation is not the same as fitting

For a fixed λ, the conditional effective degrees of freedom are $\nu(\lambda) = d + \sum_j (1 + \lambda\delta_j)^{-1}$ and normalized smoothness is (2), both computed in $O(r)$ after spectral preparation. A projected observation $z = V^\top(y - p_\mu)$ costs $O(nr)$ once, after which $\mathrm{RSS}(\lambda) = \sum_j w_j(\lambda)^2 z_j^2$ and the conditional GCV, AICc, BIC and profiled Gaussian marginal likelihood require $O(r)$ spectral sums per candidate. Classical fixed-μ LOOCV also requires leverages $(H_\lambda)_{ii}$ and residuals: calculating both using stored right singular vectors costs $O(nr)$ per penalty. This is *conditional observation-weight deletion*, not rolling-origin forecast validation, and empirical plug-in $\widehat{\mu} = \mathrm{mean}(D_d y)$ would not have the same exact leave-one-out interpretation without refitting that mean.

Nonspectral methods may fit quickly while incurring additional exact EDF and leverage work. Simply solving n banded right-hand sides to obtain inverse diagonals costs up to $O(n^2 d)$ beyond the $O(nd^2)$ factorization; specialized selected-inverse or structured algorithms can improve this (Weinert, 2007; Biessy, 2026). We distinguish the actual timed implementations from these more specialized competitors and avoid advertising a fit-only cost as an end-to-end CV-selection cost.

For dense direct methods, EDF and LOOCV may be computed using a factorization and many right-hand sides; the displayed cubic bound reflects this full operation. A properly implemented banded selected inverse is not necessarily governed by the straightforward many-right-hand-side cost. These costs exclude software-level allocation and numerical conditioning, both measured or audited separately.

2.3. How pretranslation and grid reuse change total work

With the singular values cached, one inverse $s \mapsto \lambda$ is a monotone scalar root problem in $\log\lambda$ whose function calls each take $O(r)$, so K targets with I iterations cost $O(KIr)$ beyond the SVD. Let the set $G_{n,d} = \{\Lambda_{n,d}(s_j) : 1 \le j \le K\}$ be cached. It depends neither on the current series nor on the fixed mean of differences. The same grid can be reused across C criteria and R new series.

For GCV/AICc/BIC/ML-style spectral selections, an illustrative amortized cost is

$$T_{\mathrm{SVD}} = O(nr^2 + KIr + Rnr + RCKr). \tag{4}$$

Table 1
Dominant arithmetic for the implemented algorithm families. Costs are conditional on fixed μ and known finite λ. EDF and CV entries for nonspectral methods are not interchangeable with their fit cost. We state $r = n - d$ and assume a dense spectral basis is retained.

Method	Decomposition/setup	One fit	EDF/GCV	LOOCV	Memory
Fresh dense solve	$O(nd^2)$	$O(n^3)$	$O(n^3)$	$O(n^3)$	$O(n^2)$
Cached dense solve	$O(nd^2)$	$O(n^3)$	$O(n^3)$	$O(n^3)$	$O(n^2)$
Dense Cholesky	$O(nd^2)$	$O(n^3)$	$O(n^3)$	$O(n^3)$	$O(n^2)$
General banded LU	$O(nd^2)$	$O(nd^2)$	method-specific	method-specific	$O(nd)$
Banded Cholesky	$O(nd^2)$	$O(nd^2)$	method-specific	method-specific	$O(nd)$
Dense eigendecomposition	$O(n^3)$	$O(n^2)$	$O(n)$	$O(n^2)$	$O(n^2)$
Full SVD of D_d	$O(nr^2)$	$O(nr)$	$O(r)$	$O(nr)$	$O(nr)$

The Rnr term projects each new series once; CV replaces the pertinent $O(r)$ candidate term by $O(nr)$. A cold approach that re-inverts s at every criterion or re-decomposes for each series incurs additional repeated work. Likewise, a banded Cholesky implementation that caches the same K factors across R series may incur $O(Knd^2 + RKnd)$ fitting arithmetic, *before* trace/leverage computation. This is why different setup/reuse policies can reverse a timing winner.

The reference-mean policy changes coefficients of the right-hand side, not the spectrum. Fixed $\mu = 0$ and fixed $\mu = 0.15$ share precisely the same $G_{n,d}$. Estimating $\widehat{\mu}$ from $D_d y$ adds an $O(nd)$ pass per series; the archived comprehensive Windows run did *not* separately time this data-dependent mean-estimation step.

2.4. Matched versus unmatched coordinate searches

We separate *native s grid* (invert each s_j during a search), *premapped s grid* (invert once, pass the identical λ_j to the score), and an *independently spaced log-λ grid*. Only the first two evaluate the same statistical candidates, and hence they must agree in exact arithmetic. The third may run faster but selects among *different interior penalties*; its search time alone cannot establish an efficiency advantage at equal statistical quality. Continuous bounded optimizers in s and $\log\lambda$ follow different paths and are local searches, not certified global optima. Exact $s = 0$ and $s = 1$ endpoints are scored analytically.

The classical residual-based BIC expression has a divergent interpolation boundary; our numerical companion studies this limitation. Here BIC timings are purely computational and do not imply the unrestricted criterion is statistically well-posed.

3. Archived benchmark design and provenance

This paper analyzes *completed and committed* timing experiments. It does not substitute fitted complexity curves for measured time or combine machine-specific durations from different hosts into a single speedup. The principal dataset was collected on the author's Windows 11 workstation (the environment identifies Windows build 26100) with Python 3.11.17, NumPy 2.4.6, SciPy 1.17.1, OpenBLAS limited to one thread, two experimental seeds and seven timing repetitions per reported configuration.

The full Windows source resides at `paper/benchmarks/results` with 80,640 raw rows and a separate environment record. A checked analysis yields 40,320 two-seed design cells and verifies exactly 4,484 matched native-s/premapped-grid valid seed pairs with *zero difference* in selected smoothness and score. The associated `windows_full_s_analysis/` folder preserves checks and per-cell paired costs. We use only cells whose complete evaluated grid passed the backend's recorded numerical validation; an implementation that returned a partial grid does not win a speed ranking.

3.1. Solver families and workloads

The seven Windows backends are: fresh dense solve, cached-Q dense solve, general banded solve, dense Cholesky, banded Cholesky, eigendecomposition of Q_d and full SVD of D_d. The five criteria are classical fixed-μ CV, GCV, AICc, residual-based BIC and the implemented conditional marginal-likelihood score. Eight policies cover one point at $s = 0.5$ with native/premapped/direct penalties; complete 17-point native-s, premapped and independent log-λ grids; and bounded *local* optimization in s and $\log\lambda$. All policies are evaluated at $R \in \{1, 3, 8\}$ and cold/shared preparation. Grid conversion is separately timed; a standalone premapped search charges it once, while reuse across criteria charges it only once across the entire joint workflow.

The full normalized-s grid includes the exact $s = 0$ and $s = 1$ endpoints plus near-one finite points through

Table 2
Actual coverage of the archived timing studies. The principal full-factorial rows are *not* 80,640 independent trials: they comprise 40,320 design cells and two recorded seeds. Additional repetitions are timed measurement blocks. This table separates which extra data would still be required to claim all requested combinations.

Corpus	Measured design	Limitation
Full Windows search	$n = 24, 48$; $d = 0, \ldots, 5$; $\mu = 0, 0.15$ fixed; $R = 1, 3, 8$; CV/GCV/AICc/BIC/ML; seven backends; eight search policies; cold/shared setup; 2 seeds; 7 repetitions	No $d = 6$ timing or empirical $\widehat{\mu} = \mathrm{mean}(D_d y)$
20-seed solver study	SVD, dense eigendecomposition, banded Cholesky; $M = 1, 16, 64$ full curves; $n = 32, 96, 192$ plus selected $d = 1, 3, 5$ cells; 20 independent seeds	Fit-only, not five-criterion selection
GCV reuse study	Six solver families; uniform-s versus log-λ grid; fixed means 0 and 0.15; varied grid sizes and series reuse	GCV only; generally different grid interiors
Extended fit/trace studies	Dense fresh/cached, spectral, general banded and Cholesky alternatives; selected n up to 400, M up to 256	Earlier machine/candidate regimes; not pooled with Windows factorial

$s = 1 - 10^{-12}$. For large d these correspond to potentially extreme finite penalties; failed factorizations and numerically inaccurate criterion values are explicitly tracked. The Gaussian likelihood used in this corpus was marked *not applicable* for $d = 0$. Although that historical run includes order zero, all smoothness theory and the proposed new methodological scope in this manuscript use $1 \leq d < n$.

For the representative timing table, the original script selected $n = 48, d = 2, \mu = 0, R = 1$, SVD, and the median over its two Windows seeds. The log-λ grid has 17 points but different interior values, and the continuous policies are not guaranteed global optimizers. No inferential result follows from comparing differently discretized search spaces.

3.2. Statistical interpretation of runtime measurements

A two-seed Windows timing median describes that host and work unit; it is *not* a sample of 80,640 independent processors or experiments. The separate solver study used 20 independent data seeds and paired timings to compute pointwise confidence intervals and Holm-corrected sign-flip tests for prespecified fitted-trend workloads. We report these uncertainty summaries separately rather than apply the study's p-values to the different Windows factorial. Setup, grid preparation and criterion scoring have different accounting boundaries, so their elapsed times must not be summed or divided without matching definitions.

The archived sources also do not contain an end-to-end, all-criteria $d = 6$ timing factorial, nor an additional $\widehat{\mu} = \mathrm{mean}(D_d y)$ timing policy. These are concrete extensions for the final combined paper, not silently filled gaps. The measured numerical-error companion covers $d = 6$ but cannot be converted into a runtime observation after the fact.

4. Measured computational results

4.1. Same statistical grid, different computational paths

Table 3 reproduces five *measured* standalone 17-point Windows searches, using the same SVD backend at $n = 48, d = 2, \mu = 0, R = 1$. Native-$s$ mapping and one-time pretranslation score the exact same penalties. The latter avoids repeating the scalar inversion inside every scored search, although the table charges its grid conversion once to each *independent* standalone task.

If the *same one-time candidate translation* is shared across all five criteria, the native-s total from those recorded runs was 11.234 ms, whereas grid translation paid *once* plus five backend-only searches totaled 3.980 ms: a descriptive 2.823-fold reduction for this configuration. It would be incorrect to sum the standalone premapped timings as if each were a separate grid-construction expense in this shared workflow. The effect is specific to the cached application architecture and does *not* establish that s is intrinsically faster than λ.

4.2. One evaluation, full grid and repeated independent series

Figure 3 plots all seven backends at a *single* $s = 0.5$ GCV score, one mapped 17-point GCV grid and eight independent full-grid GCV searches, under fixed $n = 48, d = 1, \mu = 0$. It uses the original full-factorial Windows rows and excludes invalid numerical results. The figure makes visible why a one-evaluation ranking need not generalize to selection or reuse workloads: decomposition cost is paid once, while data projections, curve solves and per-candidate trace work scale differently across algorithms.

4.3. Repeated grid reuse and difference order

Figure 4 shows the recorded ratio of cold to shared backend-only time for $R = 8$ complete s-grid searches across $d = 1, \ldots, 5$, conditional $\mu = 0$, $n = 48$,

Table 3
Two-seed median standalone complete-grid selection time (ms) on the recorded Windows host, one BLAS thread and seven repetitions. The native-s and premapped columns evaluate identical 17-point candidate sets; the log-λ column samples different interiors and is not a fixed-estimator speed comparison. Results are descriptive.

Criterion	Native s	Premapped $s \to \lambda$	Independent log-λ
CV	2.004	1.879	0.545
GCV	1.851	1.800	0.426
AICc	1.887	1.810	0.458
Residual BIC	2.016	1.810	0.449
Conditional marginal ML	3.477	2.035	0.515

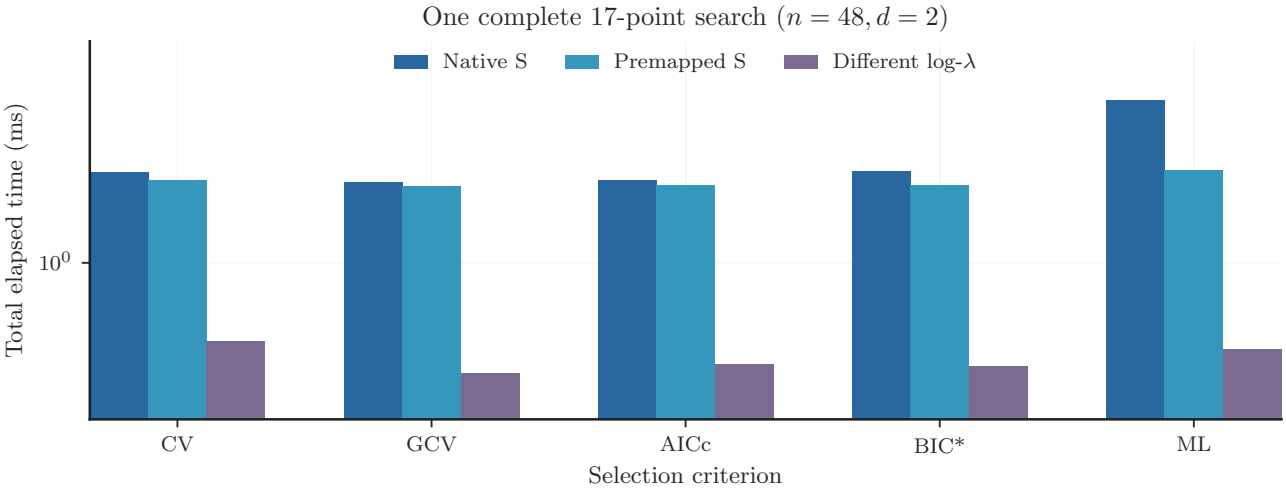


Figure 2: Completed 17-point criterion searches at $n = 48, d = 2, \mu = 0, R = 1$ with full SVD. The premapped and native-s grids contain identical candidate penalties; independent log-λ samples different interior candidates and is shown only for workload context. Log-scaled elapsed time in milliseconds; original Windows two-seed medians.

and the SVD backend. Differences among criteria are attributable to distinct scoring operations and reuse boundaries, not a change in statistical PLS formulation. A ratio greater than one favors shared setup for that cell; outcomes missing the necessary two valid seeds are left blank, not silently imputed.

4.4. Longer series and repeated fitted curves

The separate complete-trend benchmark, with n extending to 400 and up to 256 repeated fits, documents substantial changes in winners. For $n = 400, d = 2, \mu = 0$, the ratio of fresh dense to spectral total time was 0.158 at $M = 1$ but 28.314 at $M = 256$; 0.158 means the dense approach was faster for the single fit, whereas 28.314 means the spectral implementation was faster than the repeatedly fresh dense solver on the larger sweep. Against a properly general banded solver the corresponding ratios were 0.055 and 0.339, so *banded remained faster* in those two recorded workloads. These are host-specific paired cases, not universal crossovers and not five-criterion selections.

The 20-seed independent solver study also finds workload-dependent differences between full SVD, symmetric eigendecomposition and banded Cholesky. Figure 5 reports the already archived geometric mean of paired SVD/banded-Cholesky total-time ratios for $d = 2, \mu = 0$ at three n and M. Ratios greater than one favor Cholesky, less than one SVD. For example, at $n = 96$, $M = 64$, the study estimated an SVD-to-Cholesky ratio 0.959 (pointwise bootstrap interval [0.955,0.964]), while at $n = 192$, $M = 64$, it was 2.614 ([2.595,2.635]). These values and their uncertainty apply only to that prespecified fit-only design and processor. Pointwise intervals do not supply family-wise testing claims.

4.5. Exact matched-grid equivalence

The complete source audit verified 4,484 numerically valid seed-matched native-s/premapped-λ grid comparisons with *zero* differences in selected s and scored values. This is expected from (2): a monotone coordinate transform does not change the statistical model. The computational advantage lies in *the caching policy*, not statistical changes. Results from unmatched log-λ or bounded continuous search cannot establish

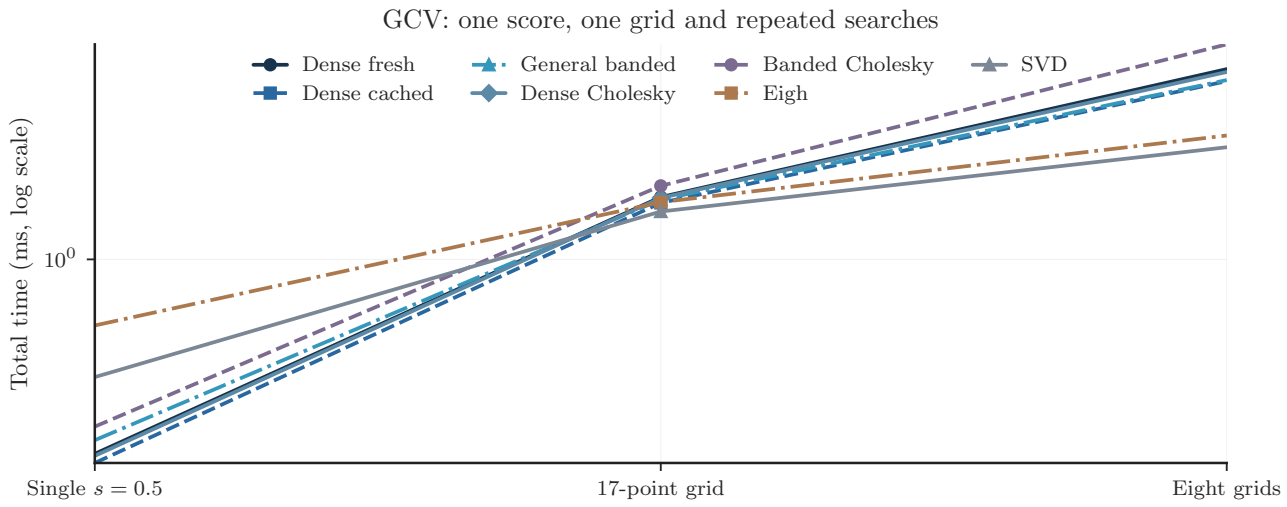


Figure 3: Observed standalone elapsed time for one GCV score ($s = 0.5$), one 17-point grid, and eight independent grids, across seven implementations. Data are from the two-seed Windows full factorial with $n = 48, d = 1, \mu = 0$ and shared preparation; only complete valid pairs are displayed. The three workload classes are distinct algorithms, not repeated scalar multiplications of one measured time.

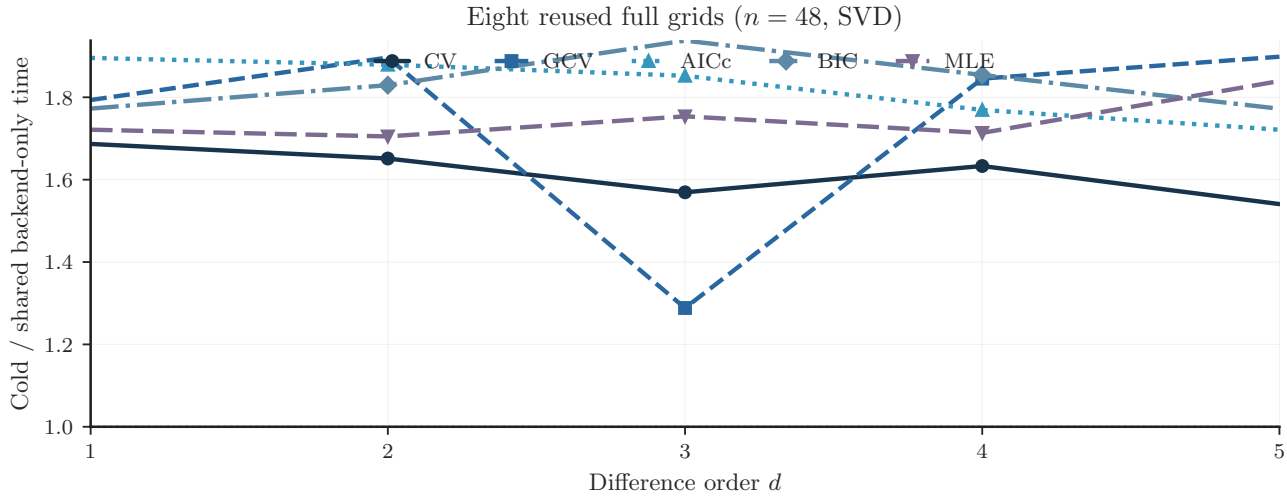


Figure 4: Cold versus shared implementation cost for eight repeated matched-grid selections by difference order and criterion. A vertical value above one indicates less backend-only time with shared preparation. $n = 48$, fixed $\mu = 0$, SVD; exact source is the archived two-seed Windows timing CSV. No $d = 6$ Windows timing was recorded.

this equivalence because they evaluate different candidate sets.

No measured cell in this section should be considered an objective benchmark at $d = 6$ or for data-derived $\widehat{\mu}$; those remain future research dimensions.

5. Discussion: when does a reusable smoothness interface pay off?

The main distinction is between *a mathematical coordinate* and *a computational policy*. The normalized smoothness map S_d is an exact reparameterization for any $1 \leq d < n$ with fixed numeric μ. It is valuable to users because the range and direction of smoothness remain interpretable across d even though penalties differ radically in magnitude. Nothing in this argument implies that optimizing in s must take less time than in λ; in fact, evaluating $s \mapsto \lambda$ at every optimizer call introduces extra computation. The measured advantage appears when the same candidate grid and spectral information are reused across calls.

Figure 2 and Table 3 demonstrate that native and premapped searches have identical statistical candidates, so their runtime differences admit a narrow, fair interpretation. The independently constructed log-λ grid is much cheaper in the representative workload,

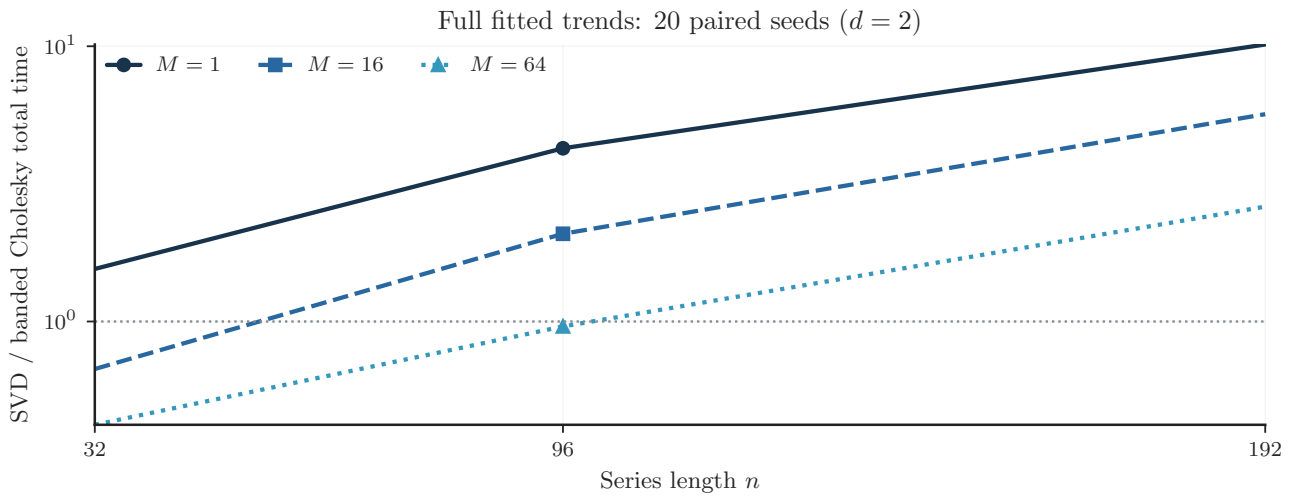


Figure 5: Paired, per-seed geometric mean SVD/banded-Cholesky *full-trend* total-time ratio at $d = 2, \mu = 0$, from the 20-seed solver study. Each line represents $M = 1, 16, 64$ requested complete fits across $n = 32, 96, 192$. A ratio below one favors SVD; above one favors Cholesky. The chart is not a direct runtime comparison of five statistical selection criteria.

but its different interior candidates can select different smoothness levels and cannot support an equal-estimator efficiency claim. Likewise bounded optimization searches are local and may differ in evaluations and objective minima; no claim of global optimization follows from their elapsed times.

The best numerical backend depends on the *actual requested output*. A banded factorization can be very economical for one fitted curve, whereas amortizing one dense SVD can accelerate repeated spectral GCV/ML scores. Exact CV leverages and specialized trace computation can reverse rankings yet again (Weinert, 2007; Biessy, 2026). High-s, large-d conditioning matters too. The independent accuracy companion found that full SVD with shift-free reconstruction is especially reliable in the tested extreme regime. However, that finding should not be back-projected as a universal wall-time speed ranking, nor should older valid low-s benchmark cells be discarded.

5.1. What the historical factorial does not establish

Although the archived Windows source is a complete factorial *within its declared axes*, it covers $d = 0, \ldots, 5$ rather than $1, \ldots, 6$ and only fixed $\mu = 0$ versus fixed $\mu = 0.15$. It does not test $\widehat{\mu} = \text{mean}(D_d y)$ as a separately timed policy. Nor does a two-seed, $n = 24, 48$ matrix establish asymptotic constants or algorithmic scaling laws at arbitrary n. We therefore report the big-O analysis independently of the machine measurements. Future work should add $d = 6$, data-derived and jointly estimated reference means, wider n, additional independent timing sessions and statistical evaluation of complete-workflow validity.

For a truly complete cross-backend selection comparison, the selected-inverse/trace approach of Weinert (2007) is essential prior art. Additional workloads should distinguish an observed-difference plug-in from fixed μ, conditional LOOCV from chronological forecast validation, independent-λ from matched-s candidate grids, and numerical failures from statistically undefined criterion values. The residual-based BIC interpolation limit should be restricted explicitly if included as a selector, as detailed by the companion reliability study.

5.2. Toward one combined manuscript

We propose ultimately combining: (i) the mathematical $s \leftrightarrow \lambda$ bijection and fixed/nonzero-μ formulas; (ii) the present algorithmic cost models, matched-coordinate parity and amortized runtime evidence; and (iii) independent high-precision solver validation and finite-sample latent-trend RMSE from the numerical companion. The comparison should use a *joint accuracy–time admissible frontier*: a solver must deliver valid equivalent criterion values over the *complete* candidate set before its timing is ranked. This avoids favoring fast, inaccurate or incomplete high-order factorizations and clarifies whether a cached spectral default is the best robust option for a specified workload.

6. Conclusions

A normalized smoothness index is a mathematically interpretable interface for quadratic finite-difference PLS at any $1 \leq d < n$, independent of the fixed mean of differences. The full singular spectrum makes the map invertible, permits reusable candidate grids and enables amortized criterion scoring. The archived

Windows benchmark demonstrates an empirical advantage from precomputing the *same* s-grid only once, with a descriptive 2.823-fold saving across five criteria in one specified SVD workload. Other complete-fit benchmarks show why banded solvers remain valuable, and no universal fastest backend follows. This computational companion establishes the complexity and runtime half of a planned combined accuracy–efficiency study, with the remaining design gaps explicitly identified.

Data and code availability

Immutable original timings, seed-level results, validations, scripts and environment metadata reside under https://github.com/heri-espino/trendestimation/tree/main/paper/benchmarks/. The original source paths and their restrictions are catalogued in this manuscript's `SOURCE_AUDIT.md`. This paper and its TeX-native figures reside in `paper/csda-computational-costs/`. No archived numerical results are overwritten or reclassified.

Declarations

Provisional authorship and affiliation are shown for drafting only. Author order, funding, conflicts and contributor statements require author approval before submission.

References

Biessy, G., 2026. Whittaker–henderson smoothing revisited: A modern statistical framework for practical use. ASTIN Bulletin 56, 1–31. doi:10.1017/asb.2025.10061.

Cortés-Toto, D., Guerrero, V.M., Reyes, H.J., 2017. Trend smoothness achieved by penalized least squares with the smoothing parameter chosen by optimality criteria. Communications in Statistics—Simulation and Computation 46, 1492–1507. doi:10.1080/03610918.2015.1005236.

Guerrero, V.M., 2007. Time series smoothing by penalized least squares. Statistics & Probability Letters 77, 1225–1234. doi:10.1016/j.spl.2007.03.006.

Guerrero, V.M., 2008. Estimating trends with percentage of smoothness chosen by the user. International Statistical Review 76, 187–202. doi:10.1111/j.1751-5823.2008.00047.x.

Weinert, H.L., 2007. Efficient computation for whittaker–henderson smoothing. Computational Statistics & Data Analysis 52, 959–974. doi:10.1016/j.csda.2006.11.038.

Whittaker, E.T., 1923. On a new method of graduation. Proceedings of the Edinburgh Mathematical Society 41, 63–75. doi:10.1017/S0013091500077853.